

Software Architecture

Exercise – Component-Oriented

BSc



Ingo Arnold

Copyright Notice

© Ingo Arnold. All rights reserved.

You may use these materials for your personal internal reference purposes only.

You may not reuse these materials in creating your own educational offerings or in your own educational situations like courses, lectures, or seminars nor may you distribute, transfer, resell or otherwise make them available to any other person or party without the expressed permission of the author.

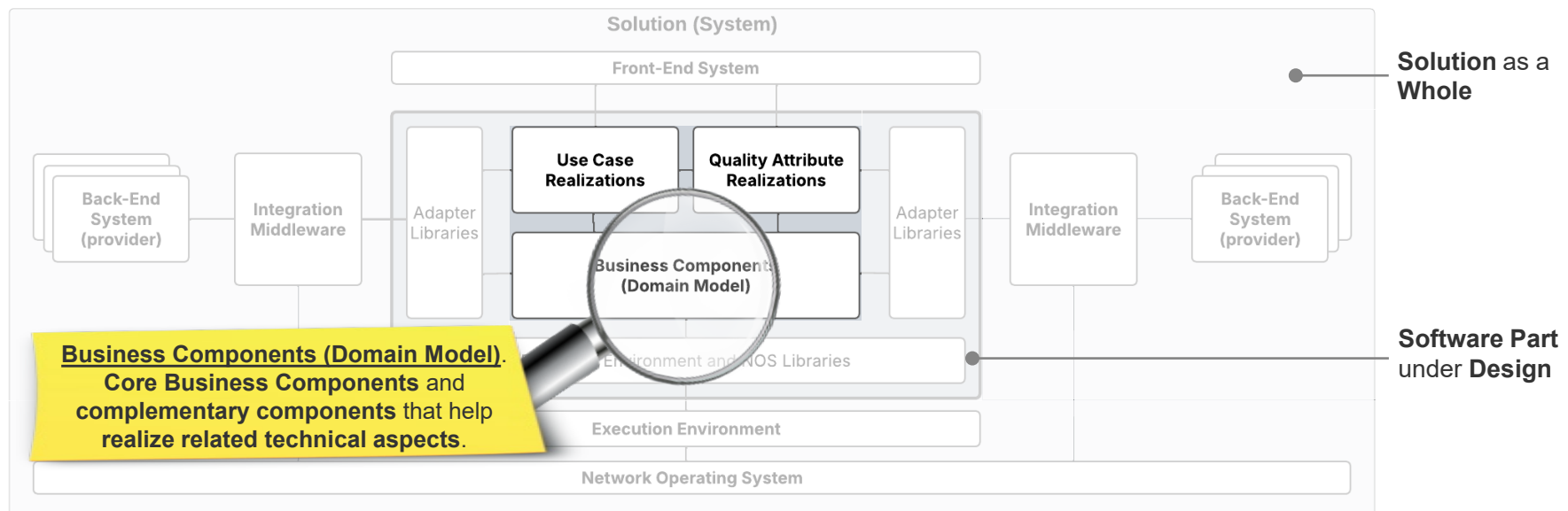
URLs or other references to Internet Web sites in the training materials are subject to change without notice. Unless otherwise indicated, all companies, organisations, domain names, email addresses, persons, places and events depicted in the Training Materials are for illustrative purposes only and are fictitious. No real connection is intended or inferred.

These training materials are provided “as is” and the author makes no warranties, express or implied, with respect to these training materials.

Exercise Opening

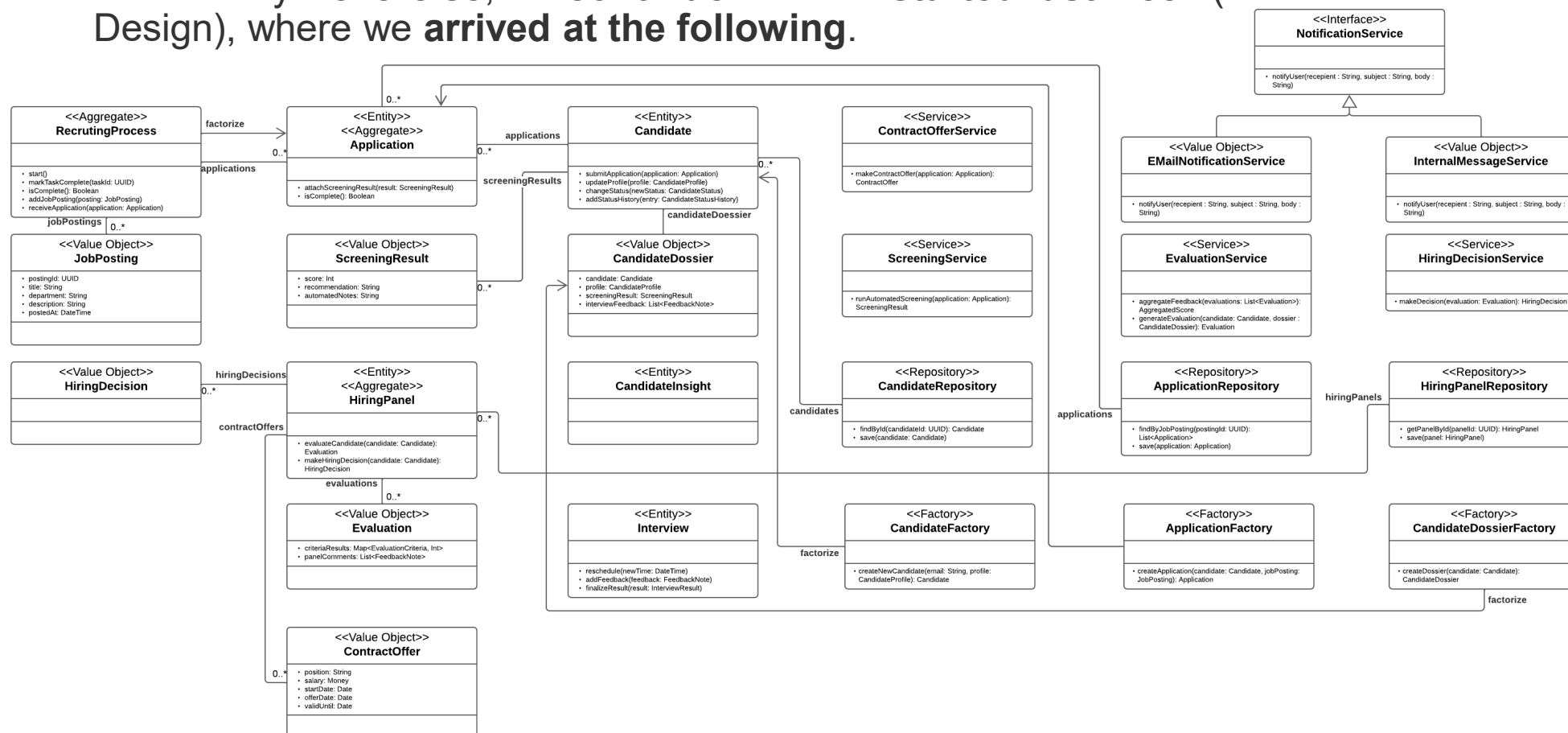
Architecture Context

In today's exercise, we will pursue **Domain-Driven Design**, with a **special focus** on addressing the **core Domain Model** and respective **Business Components**.



Exercise Opening Overview

In today's **exercise**, we **continue** what we **started last week** (Domain-Driven Design), where we **arrived at the following**.



Exercise Opening

Overview

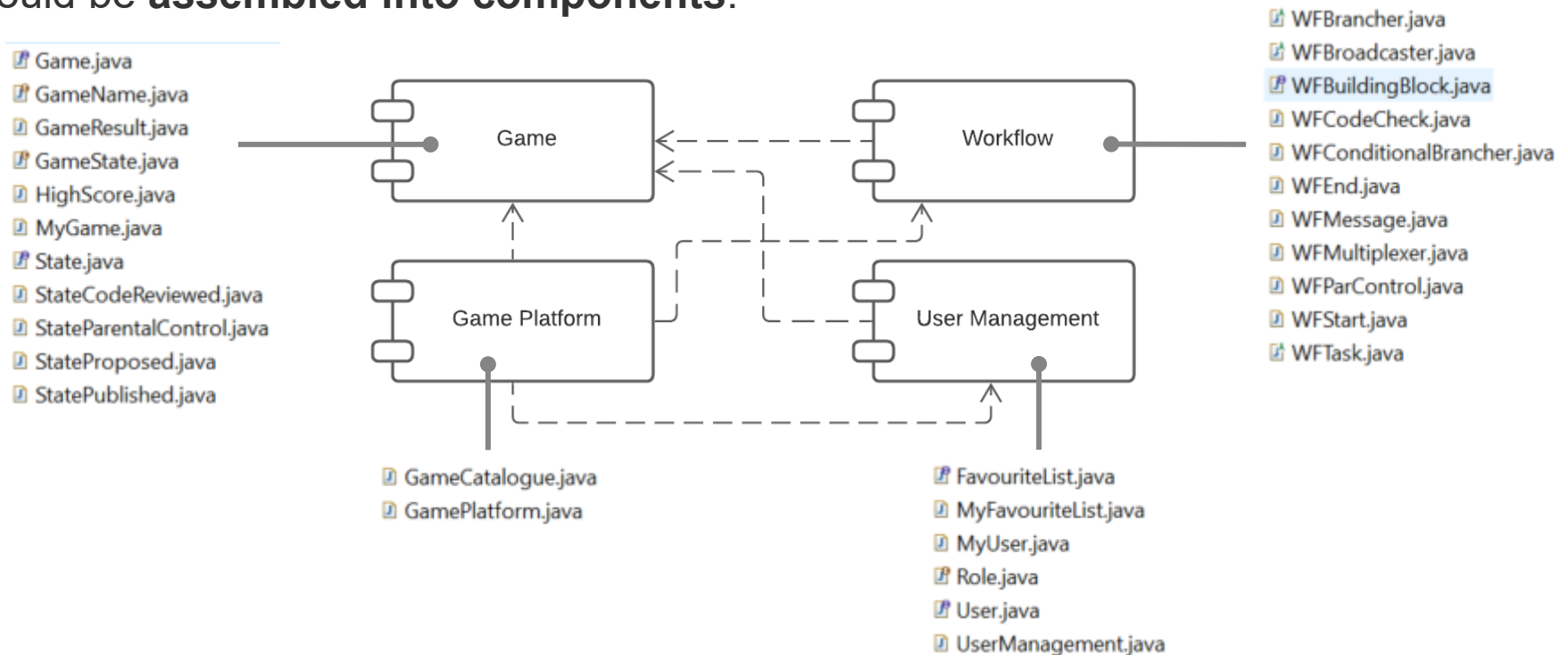
We want to **further improve** our **design** by **identifying coarser structures** (Components) and their corresponding **Interfaces** in order to **reduce** the current **complexity** (many classes).

Component Orientation

Repeating Core Concepts

Components are coarser granular than classes – in fact, components compose many classes into a larger whole.

The **example** below shows one way in which the **gaming platform classes** could be **assembled into components**.



Component Orientation

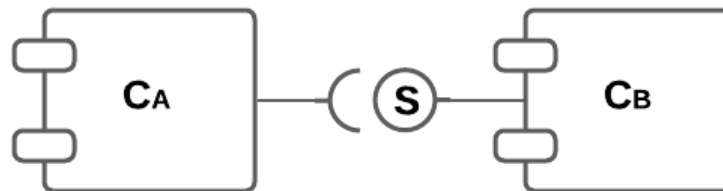
Repeating Core Concepts

Components make their capabilities available to the outside via dedicated interfaces (the providing **component exports its interfaces**).

Components use the capabilities of other components via interfaces (the consuming **component imports interfaces**).

Components bind to interfaces – not to implementations.

In the example below, **component C_B exports interface S** , while **C_A imports interface S** .

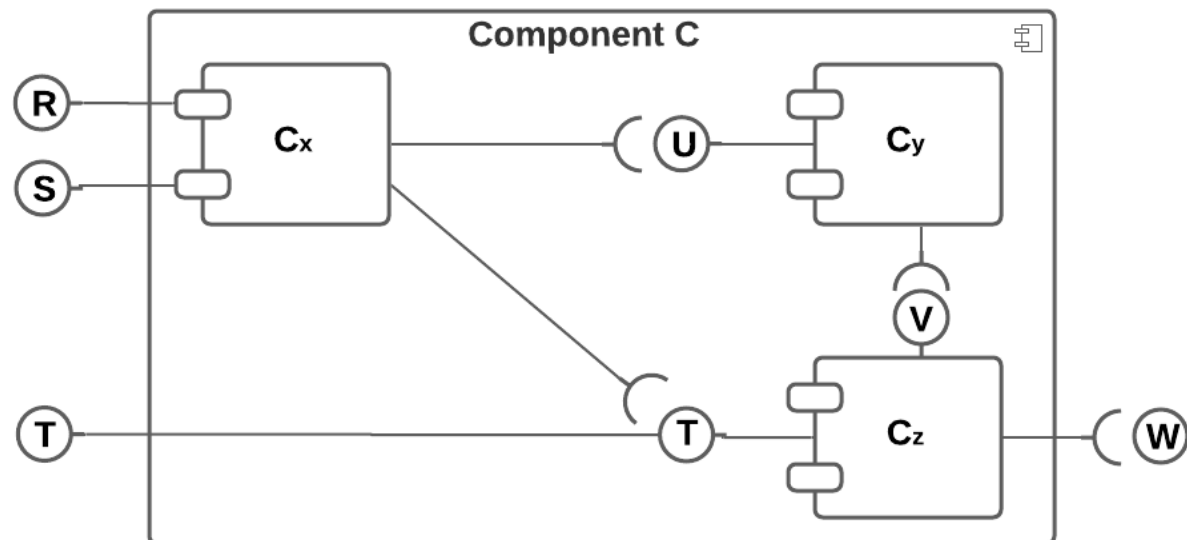


Component Orientation

Repeating Core Concepts

Components can themselves **consist of** other **components**: **Component Composition.**

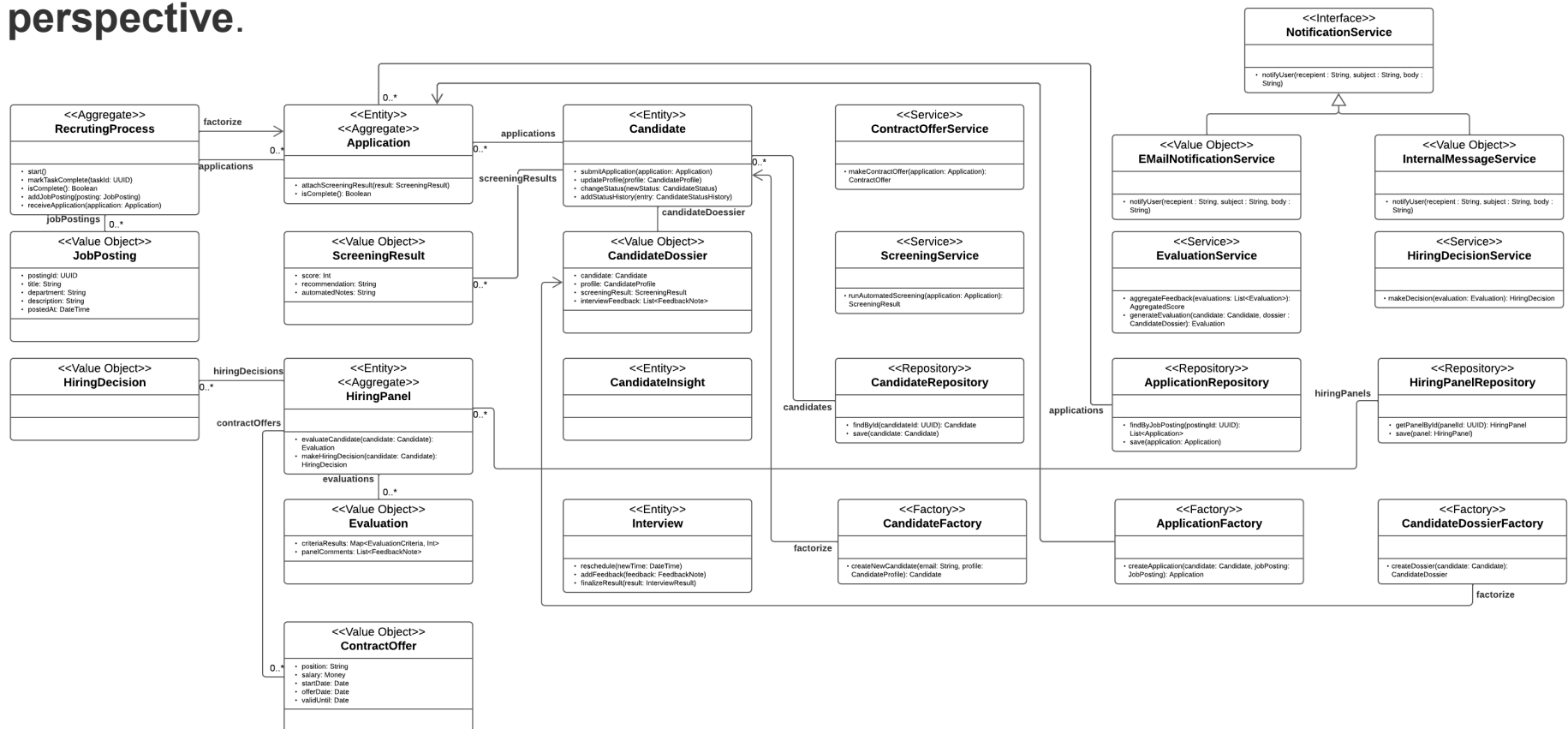
- Component C is comprised of three components C_x, C_y, and C_z.
- C_x exports interfaces R and S, while C_z exports interface T and C_y interface U.
- C_x imports interfaces U and T, while C_y imports V and C_z an external interface W.
- Component C exports the interfaces R, S, and T.



Case Study

BPO Example

In a **first step** you **analyse** the existing **class diagram** and **identify** possible **software components** that can be **clearly delineated** from a **functional/technical perspective**.



Case Study

BPO Example

Beyond the identification of Components, also elaborate (per Component) ...

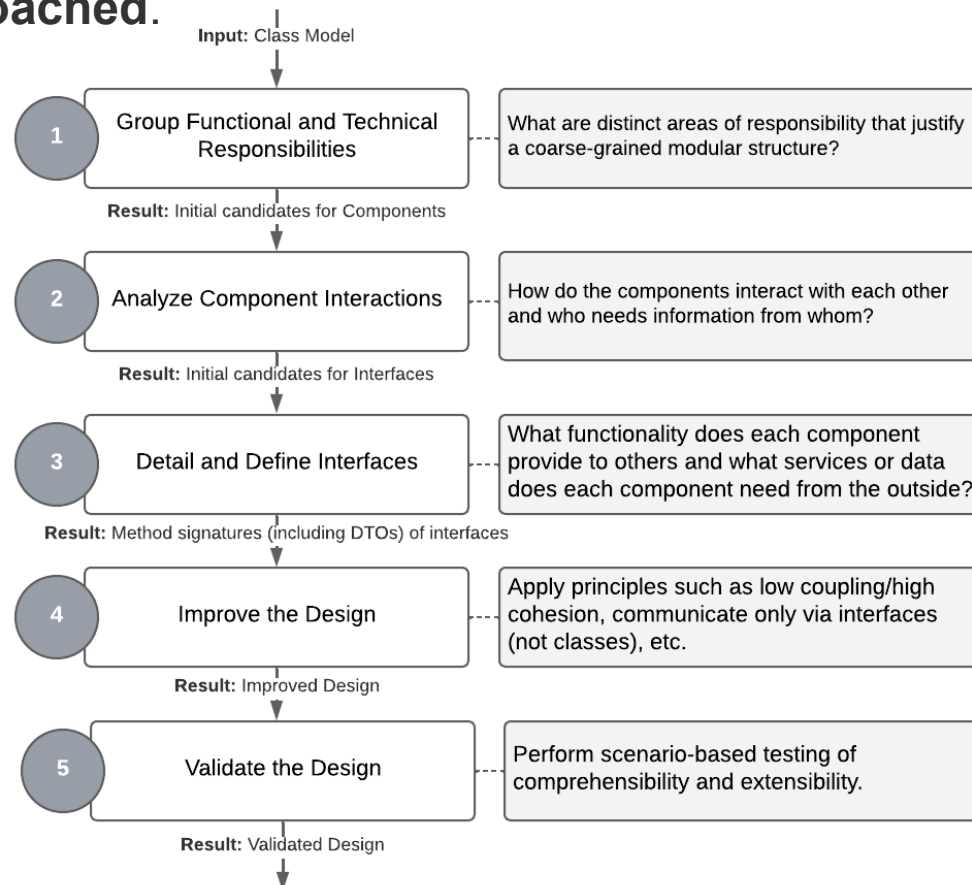
- its **responsibilities**
- its **aggregated classes**
- the **interfaces** it **provides** and its **dependencies** on other **components**



Case Study

BPO Example

Before you start solving the problem, let's look at an example to see how this is typically approached.





Case Study

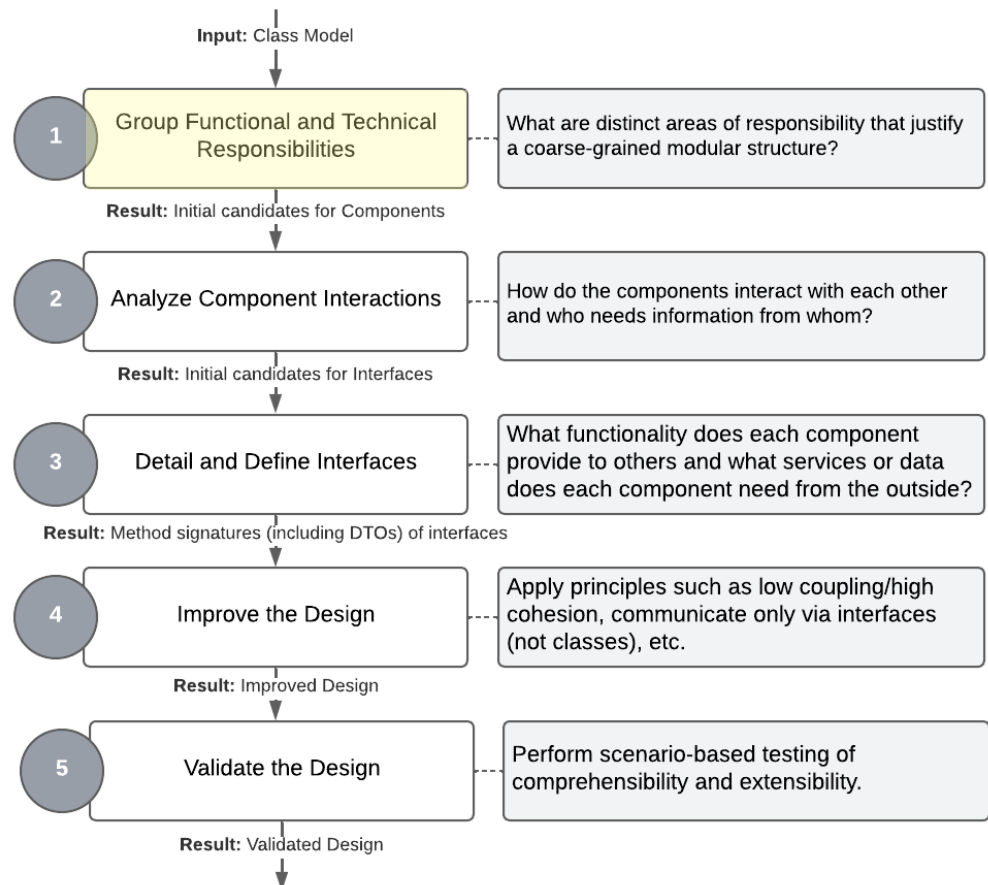
BPO Example

Before you start solving the problem, let's look at an example to see how this is typically approached.

There are a **variety of responsibilities** associated with the **Candidate domain concept**, such as ...

- **Registration of new candidates and lifecycle management of existing candidates** (CandidateFactory, Candidate, CandidateRepository).
- **Management of profile information** (CandidateProfile).
- **Change of status** (e.g., “invited,” “evaluated,” “rejected”) and logic for **status management** (CandidateStatus).
- **Data storage**, etc.

The **above responsibilities** could be **combined** in a **CandidateManagement component**.





Case Study

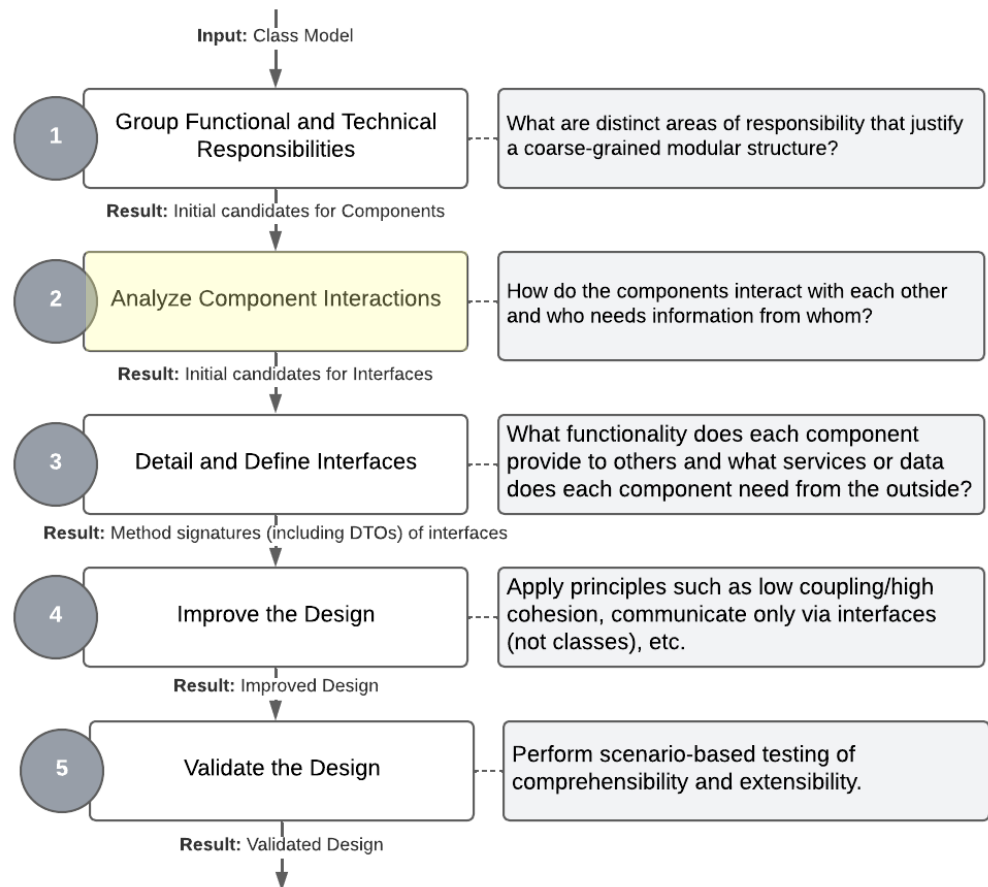
BPO Example

Before you start solving the problem, let's look at an example to see how this is typically approached.

To **analyse component interactions**, we ask ourselves **questions** such as ...

- **Who wants to register new candidates?** → Candidate component: InterviewPlanner.
- **Who evaluates candidates?** → Candidate component: EvaluationEngine.
- **Who wants to be informed about changes?** → Candidate component: NotificationDelivery.
- **Who accesses applicant data** without changing it? → Candidate component: HiringPanelSupport.

The **answers** to these **questions** give us an **initial idea** of the **component interactions** and the **corresponding interfaces between CandidateManagement and other components.**





Case Study

BPO Example

Before you start solving the problem, let's look at an example to see how this is typically approached.

From the above considerations, two interfaces are **derived**, for example, which **CandidateManagement exports** ...

- **ICandidateService** for **changes** to the **candidate object** (e.g., create, update), which contains the following method, for example:

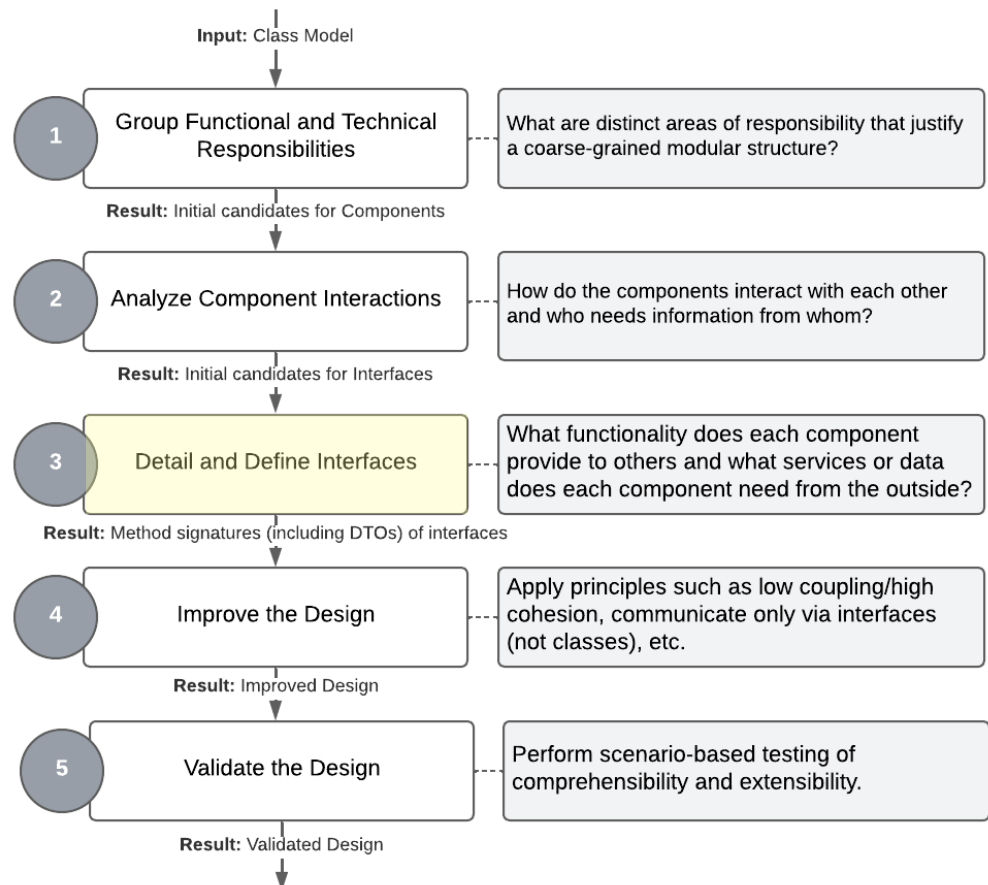

```
o void registerCandidate(CandidateProfile profile);
```
- **ICandidateQueryService** for **read-only access** (e.g., getCandidateProfile), which contains, for example, the method:


```
o CandidateProfile getCandidateById(UUID candidateId);
```

In addition, interfaces that **CandidateManagement imports** become apparent:

- **INotificationService**, which is **provided** by a **NotificationDelivery component**, for example.


```
o void notifyUser(String recipientEmail, String subject, String message);
```





Case Study

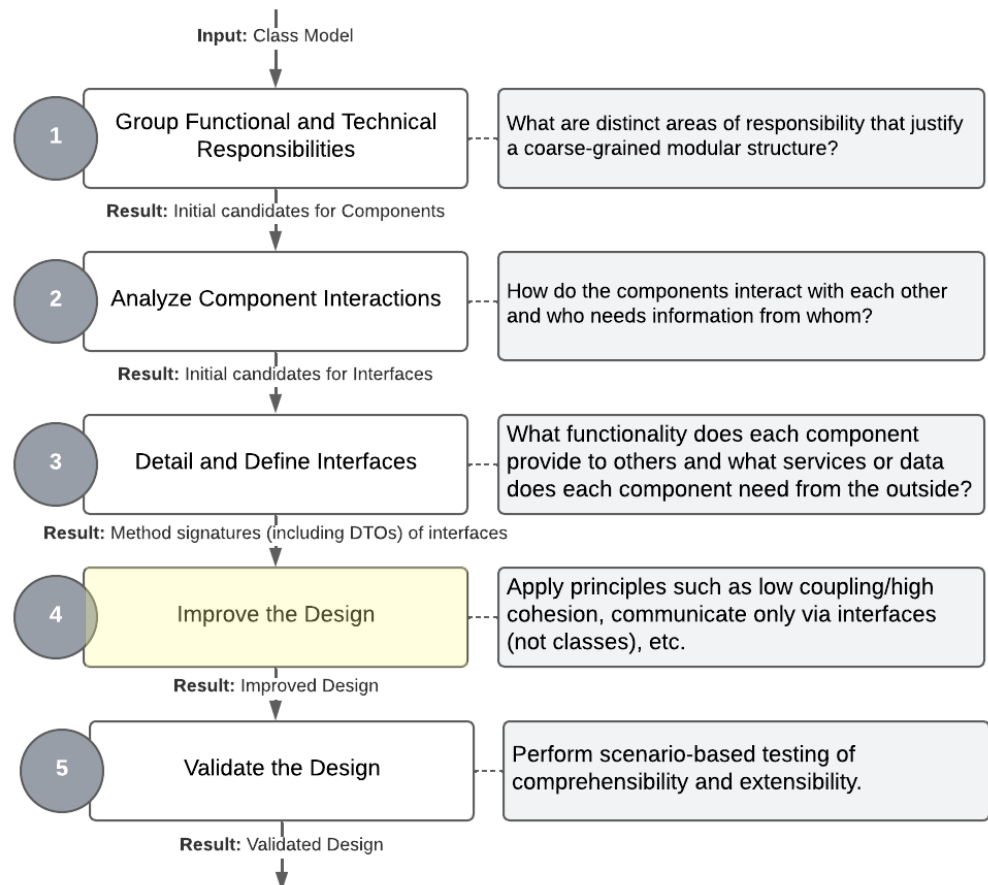
BPO Example

Before you start solving the problem, let's look at an example to see how this is typically approached.

Considerations regarding **architecture principles** and further **architecture best practices** lead to **design improvements**, such as

...

- The component **EvaluationEngine** must **not access CandidateRepository** directly, but **only the interface ICandidateQueryService**.
- The component **NotificationDelivery** is only an **event consumer**, not directly coupled.
- The component **HiringPanelSupport** only **binds to the interface ICandidateQueryService**.





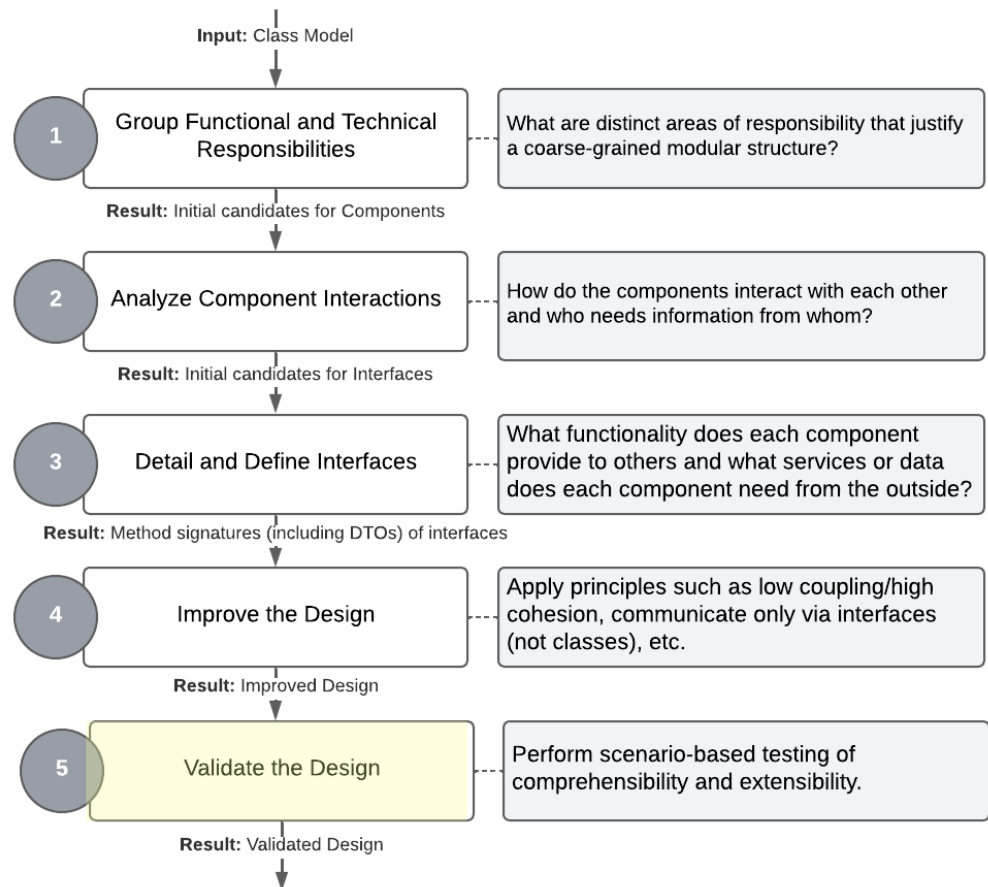
Case Study

BPO Example

Before you start solving the problem, let's look at an **example** to see how this is typically approached.

The **overall design** is ultimately **validated** and **quality-tested** based on **questions** such as ...

- Does each component have a **clear business/technical responsibility**?
- Is **data access** possible via **well-formed interfaces**?
- Is the **system easy** to **test** and **expand**?
- Are there **any unnecessary dependencies**?



Case Study

BPO Example

Beyond the identification of Components, also elaborate (per Component) ...

- its **responsibilities**
- its **aggregated classes**
- the **interfaces** it **provides** and its **dependencies** on other **components**

Now it is your turn to elaborate an
initial component design



example

Case Study

BPO Example

Here I will share my reference solution with you.

Case Study

BPO Example

Elaborate a proposal for how the `registerCandidate(CandidateProfile)` method of the `ICandidateService` interface could **be realized in a corresponding implementation** of `ICandidateService` **based on objects** of the `Candidate`, `CandidateFactory`, and `CandidateRepository` classes.

Use a diagram of type **UML Sequence Diagram** to sketch your design.



example

Case Study

BPO Example

Here I will share my reference solution with you.

Case Study

BPO Example

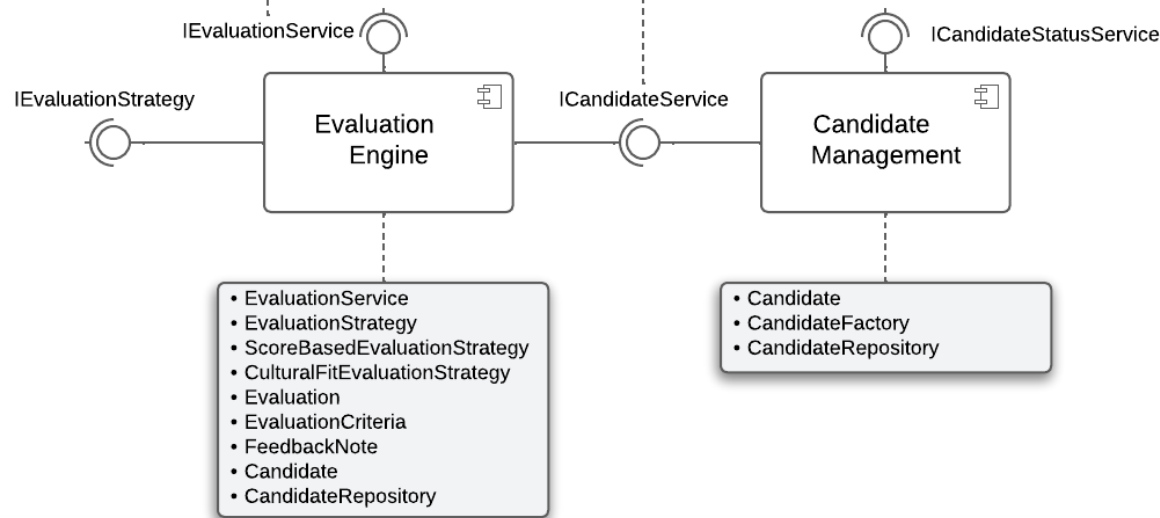
Investigate and indicate all types of coupling that exist between the CandidateManagement and EvaluationEngine components and suggest respective improvements.

```

Evaluation evaluateCandidate(UUID candidateId, CandidateDossier dossier);
List<Evaluation> getEvaluationsForCandidate(UUID candidateId);
    
```

```

void registerCandidate(CandidateProfile profile);
void updateCandidateProfile(UUID candidateId, CandidateProfile updatedProfile);
Candidate getCandidateById(UUID candidateId);
CandidateStatus getStatus(UUID candidateId);
void changeStatus(UUID candidateId, CandidateStatus newStatus)
    
```





example

Case Study

BPO Example

Here I will share my reference solution with you.

Questions

